

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет программной инженерии и компьютерной техники
Образовательная программа: 09.04.01 Компьютерные системы и технологии
Направление подготовки (специальность): Программная инженерия

О Т Ч Е Т

о научно-исследовательской работе

Тема задания: «Методология проектирования высоконагруженных систем»

Обучающийся Юнусов Ислам Рамилевич, № Р4219

Руководитель практики от университета: Ключев Аркадий Олегович, к. т. н.,
доцент, инженер.

Практика пройдена с оценкой _____

Подписи членов комиссии:

_____ Ф.И.О.

(подпись)

_____ Ф.И.О.

(подпись)

_____ Ф.И.О.

(подпись)

Дата 28.01.2026

Санкт-Петербург

2026

Оглавление

Введение.....	3
Перечень сокращений и обозначений.....	7
Основная часть.....	9
Инструктаж обучающегося.....	9
Сбор базовых материалов и определение ключевых источников информации.....	11
Анализ литературы и систематизация теоретических знаний.....	13
Формулирование исследовательской проблематики и постановка целей...	16
Разработка структуры теоретической части.....	19
Написание чернового варианта задания.....	22
Редактирование и корректировка текста.....	25
Оформление финального варианта.....	27
Оформление отчётных документов в соответствии с требованиями.....	29
Заключение.....	32
Приложение №1.....	35
Приложение №2.....	39

Введение

Растущее число цифровых сервисов, пользователей и объемов обрабатываемых данных означает, что все больше информационных систем проектируются поэтапно, даже на ранних этапах разработки. Для онлайн-магазинов, платежных платформ, социальных сетей, потоковых сервисов, а также корпоративных и государственных информационных систем критически важно обеспечение стабильной работы при миллионах запросов в день, пиковых нагрузках и жестких требованиях к времени отклика и доступности. В этих условиях архитектура высоконагруженных систем напрямую влияет на достижение требуемых показателей производительности и надежности, а также на конкурентоспособность цифровых продуктов.

Высоконагруженные системы характеризуются необходимостью одновременного решения ряда дополнительных задач, включая низкую задержку, начальную пропускную способность, горизонтальную масштабируемость, управляемые архитектурные технологии, отказоустойчивость и экономически выгодную операционную эффективность. В процессе проектирования архитектор должен учитывать стратегии масштабирования, методы распределения и хранения данных, согласованность моделей, а также подходы к обеспечению отказоустойчивости и мониторингу системы. С учётом этого решения могут основываться на ограниченном опыте отдельных команд или передовых отраслевых практиках, без формализованной методологии проектирования.

Отсутствие четко определенной методологической основы может привести к ряду системных проблем, включая фрагментированные архитектурные решения, сохраняющуюся технологическую сложность,

трудности в разработке систем и сложности в обеспечении согласованных архитектурных подходов в разных проектах. Эти проблемы особенно очевидны в контексте перехода к облачным и гибридным инфраструктурам, широкого использования микросервисов и событийно-ориентированных структур, а также использования очередей сообщений, кэширования и репликации данных. Поэтому существует необходимость не только в описании отдельных технических решений, но и в разработке комплексной методологии проектирования высоконагруженных систем, которая определяет последовательность проектных работ, критерии выбора архитектурных решений и методы их обоснования и проверки.

Целью настоящей работы является анализ, разработка и обоснование методологических подходов к проектированию высоконагруженных систем, позволяющих систематизировать архитектурные решения и уменьшить долю интуитивных и ситуативных выборов при проектировании. Для достижения указанной цели в работе решаются следующие задачи:

1. Проанализировать научные, учебные и практико-ориентированные источники, посвящённые архитектуре и проектированию высоконагруженных систем, а также методологиям разработки программного обеспечения.

2. Уточнить терминологию исследования: высоконагруженная система, высокая нагрузка, масштабируемость, устойчивость, отказоустойчивость, архитектурные паттерны и методологические подходы к проектированию.

3. Исследовать классические и современные методологии проектирования и управления разработкой (Waterfall, RUP, Agile, Scrum, Kanban, DevOps, Domain-Driven Design, Site Reliability Engineering) с точки зрения их применимости к проектированию высоконагруженных систем.

4. Выполнить сравнительный анализ методологий и связанных с ними архитектурных подходов (монолитная, микросервисная, событийно-ориентированная, cloud-native архитектуры, CQRS, Event Sourcing), выявив их преимущества, ограничения и типовые ошибки применения в условиях высокой нагрузки.

5. Сформулировать методологические рекомендации по выбору и комбинированию подходов к проектированию высоконагруженных систем с учётом их специфических требований и практического опыта индустрии.

Методологическая основа исследования включает анализ научных публикаций, диссертаций, монографий, учебных пособий, отраслевых отчётов, инженерных блогов и описаний практических кейсов компаний, разрабатывающих и эксплуатирующих высоконагруженные системы. В процессе работы применяются методы теоретического анализа и синтеза, сравнительный анализ методологий и архитектурных решений, составление конспектов и сравнительных таблиц по источникам, систематизация выявленных принципов, преимуществ, ограничений и типичных ошибок. Исследование выполняется поэтапно: от сбора базовых материалов и анализа литературы, через формулирование исследовательской проблематики и разработку структуры теоретической части, к написанию чернового варианта текста, последующей редактуре и оформлению итогового варианта в соответствии с методическими требованиями.

Практическая значимость исследования заключается в систематизации подходов к проектированию высоконагруженных систем и в разработке структурированных методологических рекомендаций, которые могут быть использованы архитекторами и разработчиками при проектировании новых решений и рефакторинге существующих. Предлагаемые обобщения и выводы позволяют снижать риски перегрузки и

отказов, повышать предсказуемость поведения системы под нагрузкой и принимать более обоснованные инженерные решения на ранних этапах жизненного цикла программного продукта.

Перечень сокращений и обозначений

1. ACID (Atomicity, Consistency, Isolation, Durability) – набор свойств транзакций в системах управления базами данных.
2. API (Application Programming Interface) – интерфейс прикладного программирования.
3. BASE (Basically Available, Soft state, Eventual consistency) – принцип проектирования распределённых систем с приоритетом доступности и «мягкой» согласованности данных.
4. CAP (Consistency, Availability, Partition tolerance) – теорема, описывающая ограничения распределённых систем по согласованности, доступности и устойчивости к разделению сети.
5. CDN (Content Delivery Network) – сеть доставки контента.
6. CQRS (Command Query Responsibility Segregation) – архитектурный паттерн разделения операций изменения данных и операций чтения.
7. CPU (Central Processing Unit) – центральный процессор.
8. DDD (Domain-Driven Design) – предметно-ориентированное проектирование.
9. DevOps (Development and Operations) – совокупность практик взаимодействия разработки и эксплуатации программных систем.
10. HTTP (HyperText Transfer Protocol) – протокол передачи гипертекста.
11. KPI (Key Performance Indicator) – ключевой показатель эффективности.
12. NoSQL (Not Only SQL) – класс систем управления данными, не основанных исключительно на реляционной модели и языке SQL.

- 13.RAM (Random Access Memory) – оперативная память.
- 14.REST (Representational State Transfer) – архитектурный стиль взаимодействия распределённых компонентов через веб-интерфейсы.
- 15.RPS (Requests Per Second) – количество запросов в секунду.
- 16.RTO (Recovery Time Objective) – целевое время восстановления системы после сбоя.
- 17.RPO (Recovery Point Objective) – целевая точка восстановления данных по состоянию на момент до сбоя.
- 18.RUP (Rational Unified Process) – унифицированный процесс разработки программного обеспечения Rational.
- 19.SLA (Service Level Agreement) – соглашение об уровне сервиса между поставщиком и потребителем услуги.
- 20.SLO (Service Level Objective) – целевое значение метрики качества сервиса.
- 21.SRE (Site Reliability Engineering) – подход к эксплуатации систем, ориентированный на надёжность и измеримые показатели качества сервиса.
- 22.SQL (Structured Query Language) – язык структурированных запросов к реляционным базам данных.
- 23.TCP/IP (Transmission Control Protocol / Internet Protocol) – стек сетевых протоколов Интернета.
- 24.UI (User Interface) – пользовательский интерфейс.
- 25.UML (Unified Modeling Language) – унифицированный язык моделирования.
- 26.СУБД – система управления базами данных.
- 27.ООП – объектно-ориентированное программирование.

Основная часть

Инструктаж обучающегося

На первом этапе обучающийся прошел вводный инструктаж, направленный на ознакомление с требованиями охраны труда и техники безопасности, технической безопасности, пожарной безопасности и внутренними правилами работы, лежащими в основе исследования.

Инструктаж охватывал основные требования охраны труда и техники безопасности при работе с компьютерами, офисным оборудованием и другим оборудованием, используемым в исследовательском процессе. Особое внимание уделялось правилам работы на рабочем месте, эргономике, графикам работы и отдыха, а также обеспечению безопасного размещения оборудования и кабельных соединений.

Инструктаж также охватывал правила электробезопасности, регулирующие подключение и отключение оборудования, требования к использованию сетевых фильтров и удлинителей, а также ограничения на вмешательство в работу электроприборов. Были также рассмотрены правила пожарной безопасности, включая порядок реагирования на признаки пожара, оповещение ответственных лиц, использование необходимого противопожарного оборудования и планы эвакуации.

Кроме того, обучающийся был ознакомлен с правилами внутреннего трудового распорядка, определяющие графики работы, процедуры управления временем, требования к дисциплине и соблюдение организационных требований, включая помещения, техническое оборудование и доступ к информационным системам.

Особое внимание было уделено необходимости соблюдения требований техники безопасности при работе с вычислительными ресурсами и сетевым оборудованием, а также обязательству своевременного информирования руководителя практики или ответственного лица при возникновении нештатных ситуаций.

Результатом данного этапа стало формирование у обучающегося понимания установленных требований по охране труда и безопасности, а также регламентов организации исследовательской деятельности в рамках базы практики. Это обеспечило необходимые условия для дальнейшего выполнения теоретической и практической частей исследования.

Сбор базовых материалов и определение ключевых источников информации

На втором этапе работы было организовано изучение теоретических и практико-ориентированных материалов, посвящённых проектированию высоконагруженных систем и методологическим подходам к их разработке. Целью данного этапа являлось формирование исходной информационной базы, необходимой для последующего теоретического анализа и построения методологической модели исследования.

На начальном этапе был выполнен поиск литературных источников в электронных ресурсах университета и специализированных научных базах данных по тематике высоконагруженных систем, масштабируемости, отказоустойчивости, архитектурных паттернов и методологии проектирования. В результате были отобраны учебные пособия и монографии, раскрывающие базовые принципы архитектуры распределённых и высоконагруженных программных систем, а также специализированные работы, посвящённые современным архитектурным подходам, включая микросервисную архитектуру и паттерны разделения ответственности при работе с данными.

На следующем этапе был проведён анализ научных статей, диссертационных исследований и материалов профильных конференций, представленных в международных индексируемых базах научных публикаций. Основное внимание уделялось работам, в которых рассматриваются методологии проектирования и управления разработкой программных систем, включая предметно-ориентированное проектирование, гибкие методологии разработки, инженерные практики DevOps и подход Site

Reliability Engineering, применительно к системам с высокой нагрузкой и повышенными требованиями к масштабируемости и надёжности.

Отдельную группу источников составили практические материалы, включающие технические отчёты, аналитические обзоры и публикации инженерного характера, описывающие опыт компаний, разрабатывающих и эксплуатирующих высоконагруженные сервисы. Анализ данных источников позволил сопоставить теоретические положения с практическими решениями, используемыми в индустрии, а также выявить условия их эффективного применения и ограничения, проявляющиеся в процессе эксплуатации.

По результатам проведённого поиска и анализа для каждого отобранного источника были зафиксированы ключевые положения, их значимость для исследования методологии проектирования высоконагруженных систем, а также отмечены основные преимущества и ограничения рассматриваемых подходов. На основе этого был сформирован библиографический список, структурированный по тематическим разделам, включающим общие вопросы архитектуры высоконагруженных систем, архитектурные паттерны, методологии проектирования и управления разработкой, а также практические кейсы и технические отчёты. Указанный перечень источников с краткими аннотациями приведён в приложении и используется в качестве основы для последующего теоретического анализа и формулирования исследовательской проблематики.

Анализ литературы и систематизация теоретических знаний

На третьем этапе исследования был проведён углублённый анализ отобранных источников, посвящённых проектированию и эксплуатации высоконагруженных систем. Цель данного этапа заключалась в систематизации ключевых понятий и подходов, связанных с высокой нагрузкой, масштабируемостью, устойчивостью и отказоустойчивостью, а также с архитектурными паттернами и методологиями проектирования программных систем.

Работа началась с уточнения и согласования терминологического аппарата. На основе анализа монографий и научных публикаций были сопоставлены различные трактовки понятий, относящихся к высоконагруженным системам, включая определения масштабируемости, отказоустойчивости, устойчивости к сбоям и эластичности. Отдельное внимание было уделено теоретическим моделям распределённых систем, включая ограничения, описываемые CAP-теоремой, принципы BASE и различные модели согласованности данных. Это позволило определить рамки, в пределах которых принимаются архитектурные и методологические решения при проектировании систем с высокой нагрузкой.

Следующим этапом анализа стало рассмотрение архитектурных стилей и проектных решений, применяемых в высоконагруженных системах. Были изучены монолитная архитектура, микросервисный подход, событийно-ориентированная архитектура и cloud-native решения, ориентированные на использование облачной инфраструктуры. На основе научных и практических источников проведено сопоставление данных подходов по таким критериям, как масштабируемость, управляемость, сложность сопровождения, устойчивость к отказам и возможности эволюции

системы. Особое внимание уделялось архитектурным паттернам, непосредственно связанным с обработкой высокой нагрузки, включая доменное разделение, паттерны CQRS и Event Sourcing, а также использование брокеров сообщений и систем очередей.

Отдельный блок анализа был посвящён методологиям проектирования и управления разработкой программных систем. На основе работ, посвящённых предметно-ориентированному проектированию, гибким методологиям разработки, инженерным практикам DevOps и подходу Site Reliability Engineering, рассматривалось влияние организационных и процессных решений на формирование и развитие архитектуры высоконагруженных систем. Проводилось сопоставление каскадных моделей разработки с итеративными и продуктово-ориентированными подходами с целью выявления их применимости, преимуществ и ограничений в условиях высокой нагрузки и изменяющихся требований.

Значимое место в рамках этапа занял анализ практического опыта компаний, эксплуатирующих системы с высокой пользовательской нагрузкой. На основе технических отчётов и инженерных публикаций были рассмотрены примеры внедрения микросервисной архитектуры, распределённых систем хранения данных, механизмов управления деградацией функциональности при перегрузке, а также организации процессов эксплуатации в рамках DevOps и Site Reliability Engineering. Это позволило сопоставить теоретические положения с практическими решениями и выявить типовые ошибки, возникающие при проектировании и эксплуатации высоконагруженных систем.

В ходе работы велись конспекты и формировались сравнительные таблицы по проанализированным источникам, в которых фиксировались общие принципы, повторяющиеся закономерности, различия в подходах и

рекомендациях, а также области применимости и характерные риски для каждой методологии и архитектурного решения.

Результатом данного этапа стала систематизированная теоретическая база, включающая согласованный понятийный аппарат, классификацию архитектурных стилей и методологий, а также обобщение практического опыта проектирования и эксплуатации высоконагруженных систем. Сформированная база используется в дальнейшем для формулирования исследовательской проблематики, уточнения целей работы и разработки методологических рекомендаций.

Формулирование исследовательской проблематики и постановка целей

На основании анализа литературы и систематизации теоретических сведений установлено, что, несмотря на значительное количество публикаций, посвящённых архитектуре и эксплуатации высоконагруженных систем, вопросы методологического обеспечения их проектирования остаются проработанными недостаточно системно. Большая часть исследований сосредоточена либо на отдельных технических аспектах, таких как архитектурные паттерны, масштабируемость и согласованность данных, либо на общих вопросах организации процесса разработки программного обеспечения. При этом комплексные методологические подходы, связывающие архитектурные решения с процессными и организационными практиками в контексте высокой нагрузки, представлены ограниченно.

Обобщение проанализированных источников позволило выявить ряд характерных противоречий. С одной стороны, классические методологические подходы, включая каскадные модели и традиционный унифицированный процесс разработки, обеспечивают формальную упорядоченность и управляемость жизненного цикла программных систем. Однако их применение оказывается затруднительным в условиях быстро изменяющихся требований, роста и изменчивости нагрузки, а также необходимости поэтапного масштабирования. С другой стороны, современные гибкие методологии и инженерные практики широко применяются в индустрии и ориентированы на эволюционное развитие систем, однако в научных и учебных источниках они зачастую описываются на концептуальном уровне, без достаточной детализации механизмов их адаптации к проектированию высоконагруженных, распределённых и облачных систем.

Дополнительная специфика проектирования высоконагруженных систем заключается в необходимости учёта факторов, которые в меньшей степени характерны для разработки традиционных веб-приложений. К таким факторам относятся профиль и неравномерность нагрузки, требования к масштабируемости и отказоустойчивости, ограничения, накладываемые моделями согласованности данных, а также особенности распределённого хранения информации и эксплуатации в облачной инфраструктуре. Вместе с тем во многих работах отсутствуют чёткие рекомендации по интеграции указанных требований в методологию проектирования и по оценке влияния организационных и архитектурных решений на устойчивость системы под нагрузкой.

С учётом выявленных противоречий исследуемая проблематика может быть сформулирована следующим образом: существующие методологические подходы к проектированию программных систем в недостаточной степени учитывают специфику высоконагруженных, масштабируемых и отказоустойчивых решений, что приводит к фрагментарности архитектурных решений и повышению рисков на этапе эксплуатации.

В связи с этим возникает необходимость разработки целостной методологической рамки, позволяющей системно увязать архитектурные паттерны, организационные практики разработки и требования, обусловленные высокой нагрузкой.

Целью настоящего исследования является разработка и обоснование методологических подходов к проектированию высоконагруженных систем, обеспечивающих более структурированный выбор архитектурных и организационных решений и снижение доли интуитивных и ситуативных решений в инженерной практике.

Для достижения поставленной цели в рамках исследования предполагается решение следующих задач:

- провести анализ научных, учебных и практико-ориентированных источников, посвящённых архитектуре высоконагруженных систем и методологиям разработки программного обеспечения, с целью выявления ключевых принципов и ограничений;
- уточнить понятийный аппарат в области высоконагруженных систем, масштабируемости, отказоустойчивости и архитектурных паттернов, а также методологических подходов к проектированию;
- исследовать классические и современные методологии разработки и управления, включая каскадные и гибкие подходы, с точки зрения их применимости к проектированию высоконагруженных систем;
- выполнить сравнительный анализ взаимосвязи методологических и архитектурных подходов, включая монолитную, микросервисную, событийно-ориентированную и облачную архитектуры, а также паттерны разделения ответственности при работе с данными, применительно к системам с высокой нагрузкой;
- сформулировать и обосновать методологические рекомендации по выбору и комбинированию подходов к проектированию высоконагруженных систем с учётом специфики нагрузки, инфраструктуры и организационных практик разработки и эксплуатации.

Таким образом, данный этап позволил уточнить фокус исследования, определить его целевые ориентиры и обеспечить логическую связь между теоретическим анализом, практическими задачами и структурой работы, реализуемой в последующих разделах. Сформулированные проблема, цель и задачи послужили основой для дальнейшего обоснования методологии и подготовки практических рекомендаций (см. «Приложение 2»).

Разработка структуры теоретической части

На данном этапе была разработана подробная и логически выстроенная структура теоретической главы, призванная системно раскрыть тему методологии проектирования высоконагруженных систем. При формировании плана учитывались выявленная исследовательская проблематика, сформулированные цель и задачи работы, а также результаты анализа литературы.

В качестве основы предложена следующая структура теоретической части:

1. Введение в проблематику высоконагруженных систем.

В данном разделе уточняются определения высоконагруженных систем, описание их ключевых характеристик (высокая интенсивность запросов, требования к масштабируемости, отказоустойчивости и эластичности), а также формулировка основных проблем при проектировании системы: необходимость учёта профиля нагрузки, ограничений распределённых вычислений, компромиссов между согласованностью, доступностью и производительностью.

2. Методологические подходы к проектированию высоконагруженных систем.

В этом блоке планируется рассмотреть классические подходы к организации разработки (Waterfall, традиционный RUP) и их ограничения при создании масштабируемых и отказоустойчивых систем, а также современные гибкие методологии (Agile, Scrum, Kanban). Отдельными подразделами выделяются инженерные практики DevOps и подход Site Reliability Engineering (SRE) как методологические рамки, связывающие

архитектурные решения с процессами эксплуатации и обеспечением надёжности.

3. Архитектурные подходы и их связь с методологиями

В рамках этого раздела структура предполагает анализ монолитных и распределённых архитектур, рассмотрение микросервисного подхода, событийно-ориентированной архитектуры, а также паттернов CQRS и Event Sourcing. Особое внимание уделяется cloud-native подходам и принципам проектирования систем для облачных сред, где методологические решения (Agile, DevOps, DDD, SRE) напрямую влияют на выбор архитектурных паттернов, стратегий масштабирования и обеспечения отказоустойчивости.

4. Сравнительный анализ подходов к проектированию высоконагруженных систем

В этом разделе планируется сопоставить различные методологические и архитектурные подходы с точки зрения их применимости к высоконагруженным системам. Структура подразумевает выделение преимуществ и недостатков каждого подхода, типичных областей применения, а также типичных ошибок при проектировании. Отдельно предусматривается включение практических примеров (кейсов компаний), демонстрирующих реальные сценарии использования методологий и архитектурных решений в системах с миллионами пользователей.

5. Методологические рекомендации по проектированию высоконагруженных систем.

Данный раздел ориентирован на формирование рекомендаций по адаптации существующих методологий под специфику высоконагруженных систем. В план включены подразделы, посвящённые комбинированию подходов, учёту требований высокой нагрузки при выборе архитектурных

паттернов, а также обсуждению перспектив развития методологий в условиях дальнейшего роста требований к масштабируемости и надёжности.

6. Заключение.

В данном блоке приведены итоги, позволяющие сделать выводы по научно-исследовательской работе.

Для каждого из перечисленных разделов были определены конкретные задачи, ключевые вопросы, подлежащие раскрытию, и ориентировочный набор литературных источников. Такая структура обеспечивает движение от общих понятий и контекста (что такое высоконагруженная система и чем она отличается от обычных веб-приложений) к более специализированному анализу методологий и архитектур, а затем — к формированию собственных методологических рекомендаций.

Результатом данного этапа стал подробный план теоретической главы, который задаёт каркас для написания чернового текста и обеспечивает логическую связность изложения. Разработанная структура согласуется с целью и задачами исследования и служит основой для дальнейшей работы над содержанием главы (см. «Приложение 2»).

Написание чернового варианта задания

На данном этапе был подготовлен черновой вариант теоретической главы в соответствии с ранее разработанной структурой, отражающей логику перехода от общих понятий к методологическим рекомендациям. Работа над черновиком сочетала обзор литературы с авторской интерпретацией и стремлением связать теоретические положения с практикой проектирования высоконагруженных систем.

Во вводной части теоретической главы были последовательно раскрыты понятия «высоконагруженная система», «высокая нагрузка», «масштабируемость», «отказоустойчивость», «устойчивость к сбоям». Приведены типичные примеры таких систем (крупные веб-сервисы, платёжные платформы, социальные сети), описаны ключевые вызовы их проектирования и эксплуатации, а также обозначена специфика высоконагруженных систем по сравнению с «обычными» веб-приложениями.

В разделе, посвящённом теоретическим основам методологий проектирования, последовательно рассмотрены классические подходы (Waterfall, традиционный RUP) и показаны их ограничения в условиях динамичной нагрузки, частых изменений требований и необходимости быстрого масштабирования. Далее изложены принципы современных методологий (Agile, Scrum, Kanban), предметно-ориентированного проектирования (DDD), а также инженерных практик DevOps и Site Reliability Engineering (SRE). Для каждого подхода описаны его ключевые идеи, область применимости и влияние на процесс проектирования и эволюции высоконагруженных систем.

Отдельный блок черновика посвящён архитектурным подходам в контексте методологии проектирования. В нём сопоставляются монолитные и

микросервисные решения, рассматриваются событийно-ориентированная архитектура, паттерны CQRS и Event Sourcing, а также cloud-native подходы. Для каждого архитектурного стиля описаны типичные преимущества и недостатки, приведены примеры практического использования в индустрии и показано, как выбор методологии разработки (Agile, DevOps, DDD, SRE) влияет на архитектурные решения.

В разделе сравнительного анализа выполнено сопоставление различных методологий и архитектурных подходов с точки зрения их пригодности для высоконагруженных систем. В черновике представлены таблицы и текстовые сопоставления, где систематизированы сильные и слабые стороны подходов, характерные риски и типовые ошибки при их применении. Особое внимание уделено вопросам согласования архитектурных решений с организационными практиками: как выбор процесса разработки и эксплуатации влияет на способность системы выдерживать рост нагрузки и сохранять надёжность.

Завершающая часть черновика содержит предварительные методологические выводы и наметки рекомендаций по адаптации и комбинированию подходов (например, использование связки Agile + DevOps + DDD + SRE при проектировании микросервисных высоконагруженных систем в облачной среде). Одновременно фиксируются вопросы, требующие дополнительного уточнения: необходимость более детальной проработки отдельных кейсов, конкретизация метрик оценки эффективности методологий, доработка примеров из практики ведущих IT-компаний.

В процессе написания чернового варианта уделялось внимание логической последовательности изложения, связям между разделами, корректному цитированию и привязке ключевых тезисов к источникам. Результатом этапа стал развернутый черновой текст теоретической главы,

включающий описание основных понятий, обзор и сравнительный анализ методологий и архитектурных подходов, а также первичные методологические выводы, подлежащие уточнению и доработке на следующих этапах редактирования и оформления.

Редактирование и корректировка текста

После подготовки чернового варианта теоретической главы был проведён этап комплексного редактирования и корректировки текста. Основная цель данного этапа заключалась в обеспечении логической целостности, полноты раскрытия темы и соблюдения академических требований к оформлению научной работы.

В ходе редактирования были выполнены следующие действия:

- **Проверка логики и последовательности изложения.**

Проанализирована структура главы на предмет плавности переходов от общих понятий к рассмотрению методологий и архитектурных подходов, а затем — к сравнительному анализу и методологическим выводам. В ряде разделов уточнены вводные и заключительные абзацы, усилены смысловые связи между подразделами, чтобы обеспечить сквозную линию рассуждений.

- **Анализ полноты раскрытия темы.**

Проверено, насколько полно в тексте отражены ключевые аспекты проектирования высоконагруженных систем: характеристики высокой нагрузки, требования к масштабируемости и отказоустойчивости, особенности архитектурных паттернов, роль методологий (Waterfall, RUP, Agile, DevOps, DDD, SRE) в проектировании таких систем. В местах, где отдельные аспекты были освещены недостаточно, добавлены пояснения, примеры и уточняющие аргументы.

- **Корректировка структуры.**

Перераспределён материал между разделами и подразделами: некоторые фрагменты, перегружавшие отдельные разделы, были перенесены в более подходящие смысловые блоки. Уточнены формулировки заголовков и

подзаголовков, чтобы они точнее отражали содержание соответствующих разделов и поддерживали единый логический каркас главы.

- **Редактура содержания.**

Устранены повторы и избыточные разъяснения, унифицированы формулировки ключевых понятий и терминов. Внесены уточнения в те места, где выводы требовали более явной аргументации или связи с приведёнными источниками. Дополнены отдельные переходные фразы для повышения читаемости текста и чёткости авторской позиции.

- **Языковая и стилистическая правка.**

Проведена орфографическая и пунктуационная проверка текста, исправлены стилистические неточности, разговорные или излишне публицистические обороты заменены на нейтральные академические формулировки. Особое внимание уделено единообразию оформления терминов, сокращений и аббревиатур, а также соблюдению норм научного стиля.

- **Проверка соответствия методическим требованиям.**

Текст сопоставлен с методическими указаниями по выполнению и оформлению научных работ. Проверены оформление ссылок, перечней и цитат, структура и нумерация разделов, соответствие требованиям к объёму и внутренней логике главы. При необходимости внесены правки в оформление библиографических ссылок и ссылки на приложения.

Результатом данного этапа стал отредактированный вариант теоретической главы, отличающийся большей ясностью, логической связностью и стилистической цельностью. Текст приведён в соответствие с академическими стандартами и методическими требованиями, что позволяет использовать его в качестве основы для финального оформления работы («Приложение 2»).

Оформление финального варианта

На заключительном этапе работы была выполнена подготовка окончательного варианта теоретической главы в соответствии с методическими указаниями и общепринятыми академическими стандартами. Данный этап был направлен на приведение текста, ссылок и структуры документа к единому, стандартному виду.

В рамках этапа осуществлены следующие действия:

- **Финальная корректура текста.**

Проведён заключительный просмотр теоретической главы с целью устранения оставшихся неточностей, уточнения формулировок и проверки целостности изложения. Особое внимание уделялось согласованности терминологии, отсутствию противоречий между отдельными разделами, а также единообразию стиля при описании методологий и архитектурных подходов проектирования высоконагруженных систем.

- **Оформление ссылок и библиографического списка.**

Все библиографические записи приведены к единому стандарту в соответствии с требованиями ГОСТ. Проверена корректность оформления ссылок в тексте, соответствие номеров источников их позициям в списке литературы, наличие указаний на электронные ресурсы с пометкой «URL» и датой обращения. Убедились, что каждое ключевое положение, заимствованное из литературы, имеет корректную ссылку на источник.

- **Форматирование документа.**

Заголовки разделов и подразделов, абзацы, нумерованные и маркированные списки оформлены в едином стиле в соответствии с методическим пособием. Проверены размеры шрифта, интервалы, поля, отступы первой строки, а также единообразие оформления таблиц и рисунков

(подписи, нумерация, расположение в тексте). Структура теоретической главы приведена в соответствие с ранее утверждённым планом.

- **Структурная проверка.**

Осуществлена проверка наличия всех обязательных элементов: введения, теоретической части, выводов по главе, списка использованных источников и ссылок на приложения. Убедился, что логика перехода от раздела к разделу сохраняется, а содержание главы полностью отвечает заявленным цели и задачам исследования, сформулированным во введении.

- **Подготовка приложений.**

Сформированы и оформлены приложения, включающие рабочий план-график, расширенный библиографический список, сравнительные таблицы по методологиям и архитектурным подходам, а также конспективные материалы по ключевым источникам. Для каждого приложения указаны наименования и обеспечена корректная нумерация, согласующаяся с основным текстом.

Результатом данного этапа стал финальный вариант теоретической главы, отличающийся целостностью структуры, единообразием оформления и соответствием требованиям методического пособия. Исследование полностью готово к представлению научному руководителю и последующему включению в итоговый отчёт по исследованию методологии проектирования высоконагруженных систем.

Оформление отчётных документов в соответствии с требованиями

Данный этап предусматривал подготовку полного отчёта в соответствии с учебно-методическим пособием и заданной структурой документа. Особое внимание уделялось тому, чтобы отчёт одновременно отражал результаты выполнения всех этапов плана-графика и строго соответствовал формальным требованиям к оформлению.

В ходе работы были выполнены следующие действия:

- **Подробное описание выполнения задач по этапам.**

В основной части отчёта последовательно описаны все этапы работы (за исключением обязательных организационных процедур): от инструктажа обучающегося до оформления отчётных документов. Для каждого этапа представлены цели, содержание выполненных действий и полученные результаты. Дополнительные материалы (конспекты, таблицы, планы, библиографические списки) вынесены в приложение.

- **Оформление отчёта в соответствии с методическим пособием.**

Оформление текста, ссылок, списков и приложений приведено в соответствие с требованиями методического пособия (<https://books.ifmo.ru/file/pdf/2622.pdf>): соблюдены параметры шрифта, интервалов, полей, правила нумерации страниц и заголовков, порядок расположения структурных элементов.

- **Соблюдение заданной структуры документа.**

Структура отчёта приведена к следующему виду:

- **Введение** – содержит обоснование актуальности темы, формулировку цели и задач исследования, краткую характеристику методологической основы и практической значимости работы.

- **Перечень сокращений и обозначений** – включает используемые в тексте аббревиатуры и термины с расшифровкой.

- **Основная часть** – состоит из последовательно расположенных разделов, отражающих этапы выполнения работы:

- *Инструктаж обучающегося;*
- *Сбор базовых материалов и определение ключевых источников информации;*
- *Анализ литературы и систематизация теоретических знаний;*
- *Формулирование исследовательской проблематики и постановка целей;*
- *Разработка структуры теоретической части;*
- *Написание чернового варианта задания;*
- *Редактирование и корректировка текста;*
- *Оформление финального варианта;*
- *Оформление отчётных документов в соответствии с требованиями.*

- **Заключение** – содержит обобщение результатов, выводы по выполненной работе и краткий анализ достижения поставленных целей.

- **Приложение** – включает дополнительные материалы, необходимые для раскрытия содержания отчёта.

- **Проверка соответствия методическим и академическим требованиям.**

Проведена итоговая проверка отчёта на предмет логической связности, полноты описания всех этапов, корректности формулировок и отсутствия разговорных оборотов. Убедился в единообразии оформления заголовков, списков, ссылок и приложений, а также в согласованности нумерации страниц со структурой документа.

Результатом данного этапа стал полностью оформленный отчёт, соответствующий установленным требованиям по структуре и оформлению и готовый к представлению руководителю и сдаче в установленной форме.

Заключение

В ходе выполнения учебно-исследовательской практики было проведено исследование, направленное на изучение и теоретическую проработку методологии проектирования высоконагруженных систем. Работа носила преимущественно аналитический характер и была сосредоточена на формировании целостной теоретической базы, необходимой для последующей разработки и обоснования методологических подходов.

На основе собранных и систематизированных источников были уточнены ключевые понятия, связанные с высоконагруженными системами: высокая нагрузка, масштабируемость, отказоустойчивость, устойчивость к сбоям, эластичность, а также роль архитектурных паттернов и организационно-процессных практик в обеспечении этих свойств. Отдельное внимание уделялось CAP-теореме, BASE-принципу, подходам к согласованности данных и их влиянию на архитектурные и методологические решения.

Проведён анализ классических методологий (Waterfall, традиционный RUP) и современных подходов (Agile, Scrum, Kanban, DevOps, DDD, SRE) в контексте проектирования высоконагруженных, распределённых и cloud-native систем. Показано, что традиционные каскадные модели разработки обладают ограниченной адаптивностью к динамике нагрузки и частым изменениям требований, тогда как гибкие методологии и инженерные практики ориентированы на эволюцию системы, тесную связку разработки и эксплуатации и управление надёжностью на основе измеримых метрик.

Сопоставление методологий с архитектурными подходами (монолитная, микросервисная, событийно-ориентированная архитектура,

CQRS, Event Sourcing, cloud-native решения) позволило выявить ключевую исследовательскую проблему: существующие методологические рамки недостаточно формализуют специфику проектирования высоконагруженных систем и часто описываются фрагментарно, без явной увязки архитектурных решений, организационных практик и требований высокой нагрузки. Это приводит к тому, что значительная часть решений принимается на интуитивном уровне, а успешные практики сложны для воспроизведения в других проектах.

В ходе практики были сформулированы цель и задачи исследования, разработана структура теоретической главы, подготовлен и отредактирован её черновой текст, а также оформлен финальный вариант в соответствии с методическими требованиями. Полученный теоретический материал представляет собой систематизированное изложение основных понятий, методологий и архитектурных подходов с акцентом на их применимость к высоконагруженным системам и на выявление типичных преимуществ, ограничений и ошибок.

Практическая ценность проделанной работы состоит в том, что сформированная теоретическая база и выделенные методологические акценты могут служить основой для дальнейшего углублённого исследования, разработки более формализованной методологии проектирования высоконагруженных систем. Сформулированные выводы и обобщения могут быть использованы архитекторами и разработчиками в качестве ориентиров при выборе методологий и архитектурных решений в проектах, ориентированных на работу под высокой нагрузкой.

Таким образом, поставленные в рамках практики цели достигнуты, запланированные задачи по этапам выполнены, а результаты работы создают

основу для дальнейшего научного и практического осмысления методологии проектирования высоконагруженных систем.

Список использованных источников

1. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – СПб.: Питер, 2018. – 640 с.
2. Эббот М. Л., Фишер М. Т. Искусство масштабируемости: масштабируемая веб-архитектура, процессы и организации для современного предприятия. – Addison-Wesley, 2015. – 624 с.
3. Найгард М. Т. Выпускайте! Проектирование и развёртывание готового к промышленной эксплуатации программного обеспечения. – Pragmatic Bookshelf, 2018. – 378 с.
4. Ньюман С. Создание микросервисов: проектирование высокодетализированных систем. – O'Reilly Media, 2021. – 612 с.
5. Ибриям Б., Хусс Р. Шаблоны Kubernetes: переиспользуемые элементы для проектирования cloud-native приложений. – O'Reilly Media, 2019. – 266 с.
6. Бейер Б., Джонс К., Петофф Дж., Мёрфи Н. (ред.) Инженерия надёжности сайтов: как Google эксплуатирует производственные системы. – O'Reilly Media, 2016. – 552 с.
7. Ким Г., Хамбл Дж., Дебуа П., Уиллис Дж. Справочник по DevOps: как создать мирового класса гибкость, надёжность и безопасность в технологических организациях. – IT Revolution Press, 2016. – 480 с.
8. Форсгрэн Н., Хамбл Дж., Ким Г. Ускорение: наука о бережливой разработке программного обеспечения и DevOps. Построение и

масштабирование высокоэффективных технологических организаций. – IT Revolution, 2018. – 288 с.

9. Эванс Э. Предметно-ориентированное проектирование: борьба со сложностью в сердце программного обеспечения. – Addison-Wesley, 2003.

10. Вернон В. Реализация предметно-ориентированного проектирования. – Addison-Wesley, 2013. – 656 с.

11. Брюэр Э. Двенадцать лет с CAP: как изменились «правила» // Компьютер. – 2012. – Т. 45, № 2. – С. 23–29.

12. ДеКандия Г., Хасторун Д., Джампани М. и др. Dynamo: высокодоступное хранилище ключ–значение компании Amazon // Труды симпозиума по операционным системам (SOSP). – 2007. – С. 205–220.

13. Лэмпорт Л. Простая реализация алгоритма Paxos // Вестник ассоциации вычислительной техники (ACM SIGACT News). – 2001. – Т. 32, № 4. – С. 18–25.

14. Онгаро Д., Оустерхут Дж. В поисках понятного алгоритма консенсуса (Raft) // Труды конференции USENIX по техническим достижениям. – 2014. – С. 305–319.

15. Фаулер М., Льюис Дж. Микросервисы: определение нового архитектурного подхода. URL: <https://martinfowler.com/articles/microservices.html> (дата обращения: 02.12.2025).

16. Фаулер М. Event Sourcing (событийное хранение состояния). URL: <https://martinfowler.com/eaDev/EventSourcing.html> (дата обращения: 02.12.2025).

17. Трифионов С. Изучаем Latency: теория массового обслуживания. URL: <https://habr.com/ru/companies/yandex/articles/431650/> (дата обращения: 02.12.2025).

18.Архитектурные паттерны в распределённых высоконагруженных системах // Хабр. URL: <https://habr.com/ru/companies/mvideo/articles/586340/> (дата обращения: 02.12.2025).

19.Архитектура высоконагруженных приложений. Масштабирование распределённых систем. Часть первая // Хабр. URL: <https://habr.com/ru/companies/badoo/articles/185220/> (дата обращения: 02.12.2025).

20.Разработка высоконагруженных API: проблемы, решения, практические рекомендации // Хабр. URL: <https://habr.com/ru/articles/928948/> (дата обращения: 02.12.2025).

21.Сложности проектирования высоконагруженных систем и как их преодолеть // Tarantool. URL: <https://www.tarantool.io/blog/ru/slozhnosti-proyektirovaniya-vysokonagruzhennykh-sistem/> (дата обращения: 02.12.2025).

22.Статистика поисковых запросов Google. – Internet Live Stats. – URL: <https://www.internetlivestats.com/google-search-statistics/> (дата обращения: 03.12.2025).

23.Amazon Prime Day 2019 – Powered by AWS // Хабр. URL: <https://habr.com/ru/articles/928948/> (дата обращения: 02.12.2025).

24.Ежедневная аудитория социальной сети «ВКонтакте» в России. – Информационное агентство ТАСС. – URL: <https://tass.ru/> (дата обращения: 03.12.2025).

25.Теорема CAP. – Википедия : свободная энциклопедия. – URL: https://ru.wikipedia.org/wiki/Теорема_CAP (дата обращения: 03.12.2025).

26.Изучаем latency: теория массового обслуживания. – Хабр (компания Яндекс). – 2018. – URL:

<https://habr.com/ru/companies/yandex/articles/431650/> (дата обращения: 03.12.2025).

27.Инжиниринг производительности систем хранения: время отклика и пропускная способность. – Хабр (компания YADRO). – 2020. – URL: <https://habr.com/ru/companies/yadro/articles/533020/> (дата обращения: 03.12.2025).

28.Мониторинг высоконагруженных систем: поддержка доступности и производительности. – Хабр (компания MONQ). – 2025. – URL: <https://habr.com/ru/companies/monq/articles/886156/> (дата обращения: 03.12.2025).

29.How DNNs Predict High-Intent Users Faster Than Ever = Как глубокие нейросети позволяют быстрее находить пользователей с высоким намерением. – Bigabid. – URL: <https://www.bigabid.com/how-dnns-predict-high-intent-users-faster/> (дата обращения: 03.12.2025).

30.What Is Throughput in Performance Testing? A Complete Guide = Что такое пропускная способность в тестировании производительности: полный обзор. – Abstracta. – URL: <https://abstracta.us/blog/performance-testing/what-is-throughput-in-performance-testing/> (дата обращения: 03.12.2025).

31.Types of Performance Testing – Everything You Need to Know = Типы тестирования производительности: всё, что нужно знать. – Abstracta. – URL: <https://abstracta.us/blog/performance-testing/types-of-performance-testing/> (дата обращения: 03.12.2025).

Методология проектирования высоконагруженных систем

1.1. Введение в проблематику высоконагруженных систем

Высоконагруженные (**highload**) системы – это ИТ-системы, на которые в короткий промежуток времени поступает чрезвычайно большой объём запросов или данных. Такие системы должны работать бесперебойно и обеспечивать минимальное время отклика даже при резком увеличении нагрузки[16]. Например, поисковая система Google обрабатывает порядка 40 000 запросов в секунду (≈ 3.5 млрд в день). Во время распродаж Amazon Prime Day сервис Amazon DynamoDB обрабатывал 7.11 трлн запросов за 48 часов с пиковым значением 45.4 млн запросов в секунду. Среди российских проектов: социальная сеть ВКонтакте с аудиторией около 100 млн уникальных пользователей в месяц и поисковый сервис Яндекса (свыше 110 млн активных пользователей поиска). Все эти системы требуют масштабируемой распределённой инфраструктуры и специальных подходов к проектированию.

Примеры высоконагруженных систем

- **Amazon (США)** – мировой лидер e-commerce и облачных сервисов. На Prime Day 2019 инфраструктура Amazon DynamoDB обработала 7.11 триллиона запросов за 48 часов, с пиком 45.4 млн запросов в секунду. Платформа Amazon Web Services обслуживает десятки миллионов клиентов по всему миру.

- **Google (США)** – крупнейший поисковик мира. По данным аналитиков, Google обрабатывает в среднем свыше 40 000 поисковых

запросов в секунду (3.5 млрд в день)[24]. Кроме того, сервисы Google (Gmail, YouTube и др.) генерируют высокую нагрузку запросов на инфраструктуру.

- **VK (Россия)** – ведущая российская социальная сеть (торговая марка VK), объединяющая разные интернет-сервисы холдинга VK. Её месячная аудитория составляет порядка 100 млн пользователей, ежедневно аудитория в России – десятки миллионов людей. Поддержка такого масштаба требует горизонтального масштабирования и специализированных БД (в том числе in-memory, например Tarantool).[21]

- **Яндекс (Россия)** – крупнейший российский IT-холдинг, включая поисковик, почту, облако, маркетплейс и др. В поисковой системе Яндекса более 110 млн активных пользователей[25]. Яндекс обрабатывает потоки запросов от десятков миллионов пользователей ежесуточно, внедряя распределённые кластеры, кэширование и отказоустойчивые архитектуры.

Каждый из перечисленных кейсов демонстрирует уникальные решения для обеспечения масштабируемости и производительности под экстремальной нагрузкой.

Основные характеристики высоконагруженных систем

Высоконагруженные системы обладают рядом специфических черт:

- **Интенсивность запросов (QPS).** Одновременно к системе поступает огромное число запросов в секунду, поэтому важна возможность быстро их обрабатывать. Например, Amazon и VK сталкиваются с десятками тысяч запросов в секунду[24].

- **Масштабируемость и эластичность.** Система должна обеспечивать эффективное горизонтальное или вертикальное масштабирование при росте нагрузки. Это включает добавление серверов, шардинг данных, использование балансировщиков, CDN и т.п. Архитектура

должна строиться модульно, чтобы можно было расширять или заменять компоненты без остановки всей системы.

- **Отказоустойчивость (надежность).** Критические компоненты проектируются без единой точки отказа (No SPOF) – дублируются и реплицируются для автоматического переключения на резерв в случае сбоя[23]. Система должна продолжать работать при отказе части узлов или оборудования, плавно деградируя производительность.

- **Целостность и согласованность данных.** При распределённой обработке огромных объёмов данных важно сохранять целостность данных. Однако из-за ограничения CAP-теоремы полная транзакционная консистентность (ACID) может быть недостижима при максимальной производительности. Поэтому во многих highload-системах применяются «мягкая» или «конечная» согласованность (eventual consistency) и компромиссные подходы (см. ниже).

- **Низкое время отклика (Latency).** Ключевая метрика – время ответа на запрос пользователя. Высоконагруженные системы оптимизируются так, чтобы латентность была минимальной[29]. Резкое увеличение задержек может привести к ухудшению пользовательского опыта и снижению конверсии, поэтому архитектура строится с учётом требований SLA.

- **Гибкость архитектуры.** Архитектура highload-систем часто складывается из микросервисов, кэшей, очередей сообщений и других компонентов, которые можно разворачивать независимо. Это позволяет гибко реагировать на изменение профиля нагрузки и вводить новые функциональные возможности без глобальных переделок[29].

Таким образом, **высокая масштабируемость, устойчивость к сбоям, низкая задержка и адекватная поддержка согласованности**

данных — ключевые требования к таким системам. Их достижение достигается за счёт распределённых архитектур, отказоустойчивых кластерами, кэширования, репликации и других техник.

Ключевые метрики

Для оценки производительности высоконагруженной системы используют следующие основные метрики:

- **QPS (queries per second)** – число пользовательских запросов или транзакций в секунду. Показатель отражает интенсивность нагрузки и пропускную способность системы. Чем выше QPS, тем большую нагрузку выдерживает сервис. Например, 45.4 млн QPS у Amazon DynamoDB в пике Prime Day[25]

- **Latency (время отклика)** – время от поступления запроса до получения ответа пользователем. Характеризует отзывчивость системы. Низкая латентность важна для удовлетворённости пользователей. Высокие задержки или резкое их возрастание могут свидетельствовать о перегрузке или неоптимальной конфигурации.

- **Throughput (пропускная способность)** – объём данных или транзакций, которые система обрабатывает за единицу времени. При нагрузочном тестировании throughput часто измеряют в запросах в секунду или мегабайтах в секунду. Высокий throughput демонстрирует, насколько эффективно система использует ресурсы.

- **Error rate** – доля неуспешных запросов. Даже при огромной нагрузке система должна поддерживать минимальный процент ошибок. Рост error rate указывает на исчерпание ресурсов или дефекты в системе.

Кроме того, важны метрики оборудования и сети (CPU, память, I/O, пропускная способность каналов) для мониторинга узких мест. Однако QPS,

latency и throughput — ключевые показатели QoS (Quality of Service) для высоконагруженных сервисов.

Теоретические основы: CAP-теорема и BASE-принцип

В основе принципов распределённых highload-систем лежит CAP-теорема Брюера. Она гласит, что в распределённой системе нельзя одновременно гарантировать все три свойства: *согласованность данных* (Consistency), *доступность* (Availability) и *устойчивость к разделению сети* (Partition Tolerance)[22]. На практике это означает: при реальных сбоях сети (разделениях между узлами) приходится жертвовать либо консистентностью, либо доступностью. К примеру, если один из кластеров недоступен, система может продолжать обслуживать запросы (с риском возвращения немного устаревших данных) или приостанавливать работу до восстановления связи (чтобы гарантировать 100%-ную актуальность данных).

Концепция **BASE** (Basically Available, Soft state, Eventually consistent) описывает компромиссные подходы в таких условиях. Эта парадигма предусматривает базовую доступность и слабое (мягкое) состояние системы с тем, что в конечном итоге данные станут согласованными. Проще говоря, система отдаёт приоритет масштабированию и доступности, позволяя кратковременное расхождение реплик. Впоследствии (без поступления новых изменений) все реплики сойдутся к последнему значению. Такой подход оправдан в сценариях, где мгновенная строгая консистентность не нужна (ленты новостей, аналитика, рекомендации), но важна высокая скорость и отказоустойчивость.

Таким образом, теоретические основы HighLoad-систем сводятся к осознанию неизбежных компромиссов: архитекторы выбирают стратегию CP (согласованность против доступности) или AP (доступность против согласованности) в момент сетевых разрывов. Знание CAP и BASE позволяет

более обоснованно проектировать базы данных и протоколы обмена данными, применяя репликацию, кворумы чтения/записи, идемпотентные операции и другие методы согласования.

Типичные проблемы при проектировании

Разработка высоконагруженных систем сопряжена с рядом серьёзных проблем:

- **Ограниченность ресурсов и баланс стоимости и производительности.** Жёсткие лимиты на CPU, память, диски и сеть требуют оптимизации. Необходимо найти компромисс между бюджетом на оборудование и требуемой производительностью. Как отмечено в отечественных исследованиях, важно создавать систему, масштабируемую без существенного роста затрат[17].

- **Масштабирование.** При росте нагрузки нужно эффективно наращивать мощности (горизонтально добавлять узлы, шарды, кэш-серверы). Неправильный дизайн (жёстко связанные компоненты, stateful-сервисы) может препятствовать масштабированию и вызывать «узкие места» производительности. Решения: безгосударственные сервисы, кэширующие слои, статические файловые хранилища и т.д.

- **Отказоустойчивость (безотказность).** В системе нельзя допускать единой точки отказа. Ключевые компоненты дублируются и реплицируются: в случае сбоя одного узла другой автоматически принимает на себя трафик. После восстановления соединения системы должны вернуться в консистентное состояние без ручного вмешательства. Это требует сложных механизмов автосделки и переключения (failover, self-healing).

- **Ограничения распределённых вычислений.** Распределённая природа влечёт сложности: узлы могут терять связь или выходить из строя, пакеты теряться или дублироваться, данные расходиться. Необходимо

учитывать сетевые сбои и временную неконсистентность (например, обеспечивать idempotency операций, таймауты и повторные запросы). Главная трудность – гарантировать целостность данных в условиях CAP-ограничений.

- **Трудности обеспечения целостности (консистентности) данных.** Сквозные транзакции (ACID) на десятках узлов зачастую невозможно сделать быстрыми. Поэтому разработчики идут на компромиссы: применяются eventual consistency, распараллеливание транзакций или специальные протоколы (согласования, двухфазный коммит). Эти меры сложны и вносят риск рассинхронизации в случае сбоев.

- **Низкое время отклика (QoS, пользовательский опыт).** Пользователи ожидают мгновенного отклика, поэтому системы оптимизируются под минимальную латентность. Даже небольшое замедление (десятки–сотни мс) может снизить удовлетворённость. Необходимо обеспечить стабильное быстроедействие при любых сценариях нагрузки. Это требует сложного мониторинга и автоматического масштабирования (auto-scaling).

Все эти проблемы повышают сложность архитектуры высоконагруженных систем. В частности, унаследованные монолиты часто оказываются не готовыми к нагрузке, что приводит к необходимости поэтапного рефакторинга, введения кэш-слоёв и новых модулей[17]. В целом, успешный проект HighLoad-системы требует глубокого анализа компромиссов (CAP vs BASE, ACID vs BASE), продуманного разделения компонентов и обширного тестирования (резилиентность, нагрузочное тестирование) на стадии разработки.

1.2. Методологические подходы к проектированию высоконагруженных систем

Методология водопадной модели (Waterfall)

Водопадная модель (Waterfall) предполагает **последовательное прохождение этапов** разработки без возврата к предыдущим фазам. Переход к следующей фазе возможен только после полного завершения предыдущей. Ключевые принципы Waterfall: детальная документация на каждом этапе, строгая фиксация требований до начала разработки и отсутствие итераций. Применяется она традиционно в **средних и крупных проектах** с десятками разработчиков, где на этапе планирования четко определены все требования и границы системы. Преимущества метода: стабильность процесса (подробная документация обеспечивает преемственность даже при смене участников), строгий менеджмент и прозрачность сроков и бюджета (все рассчитывается заранее)[10].

В контексте **высоконагруженных систем** Waterfall обеспечивает полноту проектирования на ранних фазах, но имеет значительные ограничения. Поскольку изменение требований после начала разработки обходится дорого, гибкость в адаптации архитектуры к изменяющейся нагрузке крайне мала. Проблемы производительности часто выявляются лишь на заключительном этапе тестирования, что усложняет доработки и снижает отказоустойчивость. Waterfall подходит, если нагрузка прогнозируема и бизнес-правила стабильны, однако при необходимости быстрого масштабирования и частых изменений архитектуры этот подход оказывается малоэффективным. **Примеры использования:** традиционно Waterfall применялся в проектах обороны и космической промышленности, банковском ПО и государственных системах, где требования строго

регламентированы. Сейчас же крупные компании зачастую переходят на гибридные или гибкие подходы, хотя масштабные инфраструктурные проекты с четкими спецификациями и по сей день иногда реализуются по «каскадной» схеме.

Рациональный унифицированный процесс (RUP)

RUP — это **итеративная** процессная рамочная модель, ориентированная на уменьшение рисков и акцент на архитектуру. Главные принципы RUP включают непрерывную идентификацию и устранение рисков, концентрацию на требованиях заказчика, ожидание изменений в требованиях, использование компонентной архитектуры, раннее тестирование этой архитектуры и постоянное обеспечение качества на всех этапах разработки. RUP инкрементно разбивает проект на итерации (2–6 недель), в конце каждой из которых выпускается рабочая версия продукта. Разработчики ведут визуальное моделирование, управляют изменениями и фокусируются на качественной архитектуре системы[10].

Для **высоконагруженных систем** RUP может быть применим благодаря своей гибкости и вниманию к архитектурному проектированию. Компонентный подход и непрерывная проверка качества способствуют созданию масштабируемой и отказоустойчивой архитектуры. Кроме того, многократная проверка бизнес-требований и работа в слаженной команде (с ключевой ролью архитекторов) повышают согласованность и полноту модели системы. Однако RUP нередко критикуют за **слабость адаптивности**: сильная ориентированность на документирование и процесс может замедлять внедрение изменений в условиях быстро растущей нагрузки. Кроме того, процесс внедрения RUP в крупной организации требователен к высокому уровню ответственности команды и может сопровождаться избыточным бюрократическим контролем.

Преимущества RUP для highload-архитектур: чёткая архитектура, управление рисками, формальные артефакты (технические задания, модели прецедентов, прототипы) и раннее тестирование компонентов. Это повышает прогнозируемость производительности и надёжность системы.

Ограничения: высокая стоимость разработки (множество артефактов), длительный цикл получения обратной связи и возможные не прямые расходы, связанные с управлением бизнесом, которые не относятся напрямую к производству товаров или услуг. . К примеру, компания «Рексофт» использует RUP для разработки масштабных, критически важных распределённых систем, но при этом отмечает практическое совмещение RUP с гибкими практиками в инновационных проектах.

Гибкая методология (Agile)

Agile – общее название гибких подходов к разработке. Основная идея – **разбиение больших проектов на небольшие итерации** и регулярная поставка рабочей функциональности. В Agile команда (разработчики, тестировщики, заказчики) работает параллельно и быстро реагирует на изменения требований. Документация здесь упрощена, акцент на взаимодействии и удовлетворённости клиента. Итерации (спринты) обычно длятся от 1 до 4 недель: в конце каждой достигается функциональный инкремент продукта и проводится приёмка.

В highload-разработке Agile позволяет оперативно адаптировать архитектуру и инфраструктуру под изменяющиеся требования. Быстрая обратная связь даёт возможность на ранних стадиях выявлять узкие места производительности, а перекрестное взаимодействие команд упрощает интеграцию различных частей системы. Ключевые **преимущества Agile** в контексте масштабируемости и отказоустойчивости: гибкость в планировании, частые релизы (позволяют быстро релизить фиксы и

масштабировать нагрузки) и акцент на постоянном улучшении процессов. Недостатки: отсутствие централизованного архитектурного видения может привести к техническому долгу и фрагментации системы; к тому же, при очень большом масштабе проектов синхронизация многих команд остаётся сложной задачей.

Из практики можно отметить, что большинство современных высокотехнологичных компаний (Google, Microsoft, Amazon и т.д.) используют элементы Agile. Так, по опыту ИТ-компаний, раннее определение требований и формирование backlog задач в Agile позволяет быстро реагировать на всплески нагрузки, а итеративное планирование облегчает приоритизацию работы над производительностью и отказоустойчивостью.

Scrum

Scrum – одна из самых популярных методологий внедрения Agile. Процесс Scrum строится на **фиксированных итерациях (спринтах)** продолжительностью 1–2 недели, во время которых команда регулярно поставляет программный продукт. Роль Product Owner приоритизирует беклог, Scrum Master следит за процессом, а самоорганизованная команда разработки реализует задачи. В конце каждого спринта проводится обзор результатов и ретроспектива для анализа эффективности процесса.

В контексте высоконагруженных систем Scrum полезен тем, что обеспечивает регулярное планирование и своевременную проверку промежуточных результатов. Имеется возможность встраивать в итерации задачи по нагрузочному тестированию и оптимизации производительности.

Преимущества Scrum: высокая прозрачность работ, чёткое распределение ролей и обязательность демонстрации работающего продукта каждый спринт.

Ограничения: архитектурное планирование может пострадать из-за коротких спринтов (нужны дополнительные обсуждения об архитектуре вне

самых спринтов); масштабирование Scrum на большие проекты требует специальных практик. Для крупных проектов применяют «Scrum of Scrums», когда представители нескольких команд синхронизируют работу и решают межкомандные зависимости.

Примеры: Scrum-схему используют в IT-компаниях разного размера. Например, многие продукты компании Atlassian (разработчика Jira и Confluence) разрабатываются командами по Scrum. Часто встречаются гибридные модели («Scrum-ban» и др.), когда рутинные задачи в поддержке системы управляются по Kanban, а новые фичи – по Scrum.

Kanban

Kanban – методология управления потоком задач, акцентирующая внимание на непрерывном **визуализации работы и ограничении незавершённых задач (WIP)**. Канбан-доска отражает текущий процесс («К выполнению», «В работе», «В тестировании», «Выполнено» и т.д.), что позволяет всей команде видеть статус каждой задачи. Основные принципы: визуализировать поток работ, ограничивать WIP, управлять и улучшать процесс, чётко определять критерии готовности и вносить малые постоянные изменения.

Для **высоконагруженных систем** Kanban особенно полезен при поддержке и эксплуатации. Отслеживание инцидентов и небольших улучшений через доску позволяет быстро реагировать на проблемы производительности и надежности. Канбан не имеет фиксированных итераций, поэтому новые запросы (например, заявки на масштабирование ресурсов или оптимизацию узких мест) попадают в поток задач по мере поступления. **Преимущества:** гибкость процессов, отсутствие бюрократической перегрузки, постоянное улучшение. Благодаря

ограничению WIP Kanban обеспечивает более высокую скорость и гибкость команды, минимизируя многозадачность.

Ограничения: отсутствие сроков может приводить к нерегулярному релизу больших улучшений, возможна нехватка фокуса без строгого управления.

Примеры: Kanban пришёл из промышленности (Toyota) и широко применяется в IT-операциях. Многие сервисные и DevOps-команды крупных компаний (например, Microsoft Azure или Яндекс.Директ) используют Kanban-доски в Jira/Confluence для управления потоком инцидентов и задач по оптимизации. Так, внутри компании Atlassian поддержка своих сервисов ведётся часто именно по Kanban.

DevOps

DevOps – методология автоматизации технологических процессов сборки, настройки и развёртывания программного обеспечения. Термин складывается из двух слов: разработка (Dev) и операционная деятельность (Ops). Основная цель DevOps – ускорить поставку ПО и повысить его качество за счёт автоматизации процессов развертывания и интеграции, а также тесного взаимодействия команд. Для высоконагруженных систем DevOps особенно актуален: он помогает сократить время простоя, повысить устойчивость системы и оперативно масштабировать инфраструктуру.

Ключевые элементы DevOps: **CI/CD** (непрерывная интеграция и поставка), **контейнеризация и оркестрация** (Docker, Kubernetes), **инфраструктура как код** (Terraform, Ansible), а также автоматизированный **мониторинг и логирование**. Например, автоматизация развёртывания и тестирования облегчает выпуск обновлений с минимальными сбоями, что критично при высокой нагрузке. Контейнеры и оркестраторы обеспечивают создание изолированных, масштабируемых сред; это позволяет **эффективно**

распределять ресурсы и обеспечивать высокую доступность сервисов при резких пиковых нагрузках.

Преимущества DevOps в контексте highload: быстрая и регулярная поставка обновлений, автоматическое масштабирование сервисов, всесторонний мониторинг (что позволяет мгновенно реагировать на сбои). Так, практика непрерывного наблюдения (Prometheus, ELK и др.) минимизирует время простоя системы. **Ограничения:** внедрение DevOps требует изменения организационной культуры и навыков команды; ошибки в автоматизированных процессах могут быстро распространиться. Кроме того, большая гибкость и множество автоматических интерфейсов требуют повышенного внимания к безопасности на всех этапах.

Примеры: DevOps является стандартом «культуры» для таких гигантов, как Netflix, Amazon, Google. В частности, Netflix известен инженерным подходом к надёжности (Chaos Engineering) – автоматизированными сценариями аварий (Chaos Monkey) он проверяет отказоустойчивость системы. Большинство крупных IT-компаний отмечают, что именно DevOps-практики (CI/CD, автоматическое масштабирование и т.д.) позволяют им управлять сверхсложными высоконагруженными сервисами.

Domain-Driven Design (DDD)

DDD – не методология управления проектом, а **подход к проектированию ПО**, фокусирующийся на модели предметной области. Он предлагает создавать **богатые доменные модели**, отражающие бизнес-правила и логику предметной области в коде. Центральные концепции DDD: **единый язык** (Ubiquitous Language), **ограниченные контексты** (Bounded Context) и тактические шаблоны (сущности, агрегаты, службы). Цель DDD – получить высококачественную модель системы, максимально

точно отражающую бизнес-цели, путем тесного сотрудничества разработчиков и экспертов предметной области.

В высоконагруженных системах DDD помогает **структурировать систему на независимые модули (сервисы)** по бизнес-контекстам. Каждая область может масштабироваться и деплоиться отдельно, что повышает отказоустойчивость и гибкость архитектуры. Например, в интернет-магазине можно выделить контексты «Управление заказами», «Оплата», «Поиск», и разворачивать их в отдельных микросервисах – это снижает взаимное влияние при пиковых нагрузках. **Преимущества DDD:** чёткое разделение ответственности, снижение связности систем, лучшая управляемость сложностью. Такой подход способствует масштабированию, так как позволяет концентрировать ресурсы на наиболее важном домене и оптимально распределять нагрузку между частями системы. **Ограничения:** DDD требует глубокого погружения в домен и сложен в применении («тонкая» настройка модели). Неправильное выделение контекстов или чрезмерный формализм могут привести к усложнению архитектуры и росту времени разработки.

В практике разработки высоконагруженных сервисов DDD часто совмещают с микросервисной архитектурой. Как отмечалось на конференции HighLoad++, DDD вместе с CQRS и микросервисами становятся стандартом при создании современных крупных систем, обеспечивая **масштабируемость и целостность данных**. Крупные компании (например, Amazon, Netflix) применяют DDD-принципы для проектирования своих распределенных систем, где каждая бизнес-область развивается изолированно, а интеграция происходит через чётко определенные API.

Site Reliability Engineering (SRE)

SRE – инженерный подход к эксплуатации сервисов, популяризированный Google. Он использует ПО для автоматизации рутинных операций и **гарантирует надёжность ПО** при частых обновлениях. Суть SRE – управлять надёжностью через код, а не через бесконечные операционные дежурства. В SRE выделяют метрики SLA/SLO и «бюджет ошибок»: команде разрешено определённое количество инцидентов, что стимулирует искать баланс между выпуском фич и стабильностью.

По определению AWS, SRE «исследует варианты использования ПО-инструментов для автоматизации задач IT-инфраструктуры», чтобы **программы оставались надёжными несмотря на частые обновления команд разработчиков**. Особенно SRE улучшает надёжность **масштабируемых систем**, поскольку управление сотнями или тысячами серверов вручную менее эффективно, чем автоматизированные процессы. Ключевые принципы SRE: постоянный мониторинг и алертинг (SLI/SLO), постепенное развёртывание изменений (микро-релизы), автотестирование релизов и автоликвидация инцидентов.

Для высоконагруженных систем SRE приносит критические **преимущества**: превентивное планирование отказов, быстрое восстановление после сбоев, автоматизация масштабирования и чёткий подсчёт стоимости простоя. Команды SRE активно используют инфраструктуру как код, динамическое масштабирование и катастроф-тестирование, что напрямую повышает отказоустойчивость системы. **Ограничения**: требует наличия узкоспециализированных инженеров и подготовки команды к работе с SLA. Переход к SRE означает пересмотр подхода к управлению изменениями – организации должны быть готовы к культурным изменениям (например, к безнаказанным post-mortem). Впрочем, опыт компаний вроде Google и Microsoft показывает, что внедрение

SRE значительно повышает надёжность крупномасштабных сервисов в долгосрочной перспективе[14].

Сравнительный анализ

Классические модели (Waterfall, RUP) ориентированы на **предсказуемость и формальный контроль**. Waterfall проста для понимания и хорошо документирована, но плохо адаптируется к изменениям нагрузки: все проблемы производительности выявляются поздно. RUP более гибок благодаря итерациям и упору на архитектуру, что помогает прогнозировать масштабируемость, однако имеет значительный управленческий оверхед.

Современные гибкие методологии (Agile, Scrum, Kanban) фокусируются на **быстром отклике на изменения и непрерывном улучшении**. Agile и Scrum позволяют делить разработку на короткие циклы, что ускоряет разработку оптимизаций производительности, но требуют дополнительной дисциплины для сохранения целостности архитектуры. Kanban обеспечивает максимальную гибкость и подходит для непрерывной поддержки системы, но без жёстких дедлайнов может порождать непредсказуемость релизов.

Инженерные практики DevOps, DDD и SRE дополняют методы управления фактически **техническими инструментами**. DevOps автоматизирует инфраструктуру и развёртывание, что критично для масштабируемых систем: CI/CD, контейнеризация и мониторинг повышают отказоустойчивость и скорость реакции на нагрузку. DDD структурирует систему по бизнес-контекстам, облегчая масштабирование отдельных подсистем и снижая связность, что положительно влияет на гибкость и надёжность архитектуры. SRE же задаёт инженерные рамки надёжности (SLO и автоматизация), обеспечивая устойчивость сервисов даже под экстремальными нагрузками.

Таким образом, при проектировании **highload-архитектур** чаще используют гибридный подход. Так, базовая методология разработки может быть гибкой (Agile/Scrum), RUP-практики интегрированы в критических частях, а DevOps/SRE-инструменты – в процессы эксплуатации. В результате Waterfall/RUP дают надёжность и планируемость, Agile/Scrum/Kanban – быструю адаптивность, а DevOps/DDD/SRE – техническую устойчивость и масштабируемость. Каждая методология дополняет остальные: идеальная практика может содержать итеративный цикл разработки с учётом архитектурных требований (RUP/DDD), непрерывную поставку и автомасштабирование (DevOps/SRE), и гибкость взаимодействия с заказчиком (Scrum/Kanban). Источники свидетельствуют, что комбинированное применение подходов с учётом их сильных сторон позволяет наиболее эффективно справляться с вызовами высоконагруженных систем.

Сравнительная таблица (кратко):

- *Масштабируемость*: наименее гибко – Waterfall; RUP и Agile дают среднюю гибкость; DevOps и DDD значительно повышают масштабируемость за счёт модульности и автоматизации; SRE ориентирован на поддержание масштабируемости при росте нагрузки.
- *Отказоустойчивость*: Waterfall/RUP зависят от полноты тестирования на завершающих этапах; Agile/ Scrum/Kanban полагаются на быстрый цикл исправления дефектов; DevOps обеспечивает отказоустойчивость через контейнеризацию и мониторинг; SRE – за счет SLA-политик и автоматического восстановления.
- *Гибкость*: Waterfall минимальна; RUP средняя (итерации); Agile/Scrum/Kanban – очень высокая; DevOps/DDD/SRE повышают гибкость системы в техническом плане.

Каждая методология имеет свои сильные стороны в контексте **highload**: сочетание их в рамках реальных проектов (например, гибрид Agile+DevOps+DDD) позволяет учесть и скорость разработки, и масштабируемость, и надёжность итоговой системы[12].

1.3. Архитектурные подходы и их связь с методологиями

Монолитная архитектура

Монолитная архитектура представляет собой традиционный подход к построению ПО, при котором все компоненты приложения объединены в один единый исполняемый модуль (единую кодовую базу). Это упрощает разработку и тестирование: единый код и общая БД обеспечивают высокую скорость разработки и простоту деплоя. Однако монолит плохо масштабируется и надёжность системы ограничена: ошибка в одном модуле может вывести из строя весь сервис. По мере роста нагрузки приходится масштабировать всё приложение целиком (например, добавлять реплики и балансировщик). В итоге крупные монолиты становятся негибкими и медленными в изменениях: чтобы внести небольшое изменение, требуется пересобрать и развернуть всю систему. Монолит часто применяют на начальных этапах разработки (proof-of-concept, MVP) или в небольших проектах с фиксированным функционалом. Традиционно для монолитов характерны классические методологии разработки (Waterfall, RUP) и строгие процессы, тогда как быстрая итеративная методика Agile/Scrum затруднена из-за больших временных затрат на сборку и деплой целиком.

- **Особенности и преимущества:** единая кодовая база, простота разработки и сквозного тестирования, эффективный внутренний обмен данными, простота развертывания.

- **Недостатки:** низкая гибкость, трудности с внедрением новых технологий, масштабированием и обновлениями (любой сбой в модуле влияет на весь сервис).

- **Методологии разработки:** традиционно сочетается с классическими подходами (Waterfall, RUP) – задачу архитектуры

проектируют сразу целиком, – тогда как гибкие методики (Agile/Scrum) затрудняются: обновление одной функции требует пересборки всего приложения, что противоречит принципам непрерывной поставки.

- **Отказоустойчивость и масштабирование:** монолит сам по себе мало устойчив к отказам (неразделённое приложение), для повышения отказоустойчивости применяют реплики и балансировку. Горизонтальное масштабирование возможно только полным копированием приложения; эластичность ограничена (обычно используют вертикальное масштабирование, добавляя мощности серверу).

- **Примеры:** многие старые системы (банковские платёжные и ERP/CRM-системы) и legacy реализованы как монолиты. Так, Twitter изначально был монолитным приложением, но по мере роста нагрузки столкнулся с проблемой масштабирования (сложностью горизонтального роста), что и послужило поводом перехода на другую архитектуру. Аналогично eBay и многие корпоративные системы изначально строились как монолиты.

Микросервисная архитектура

Микросервисная архитектура делит систему на множество мелких независимых сервисов, каждый из которых реализует свою бизнес-функцию и обычно имеет собственную БД и API. Это облегчает управление сложностью: сервисы изолированы друг от друга, обладают слабой связностью, и каждый разрабатывается и разворачивается автономно. Такой подход тесно связан с практиками DevOps и методиками CI/CD (непрерывной интеграции и доставки), благодаря чему команды могут быстро вносить изменения и часто разворачивать новые версии отдельных сервисов. Также микросервисы хорошо сочетаются с Domain-Driven Design: каждый сервис часто отражает отдельный «bounded context» предметной области.

- **Особенности:** независимость сервисов, эффективные сервисы, но сложные каналы взаимодействия (через REST, сообщения и события). Сервисы ориентированы на одну задачу, легко заменяются и могут быть реализованы на разном стеке технологий. Это повышает гибкость разработки и облегчает распределение команд по функциональным областям.

- **Методологии разработки:** микросервисы обычно строятся в рамках Agile/Scrum с DevOps-культурой. Поскольку каждый сервис небольшой, часто применяется непрерывная поставка и автоматизация инфраструктуры (Docker, Kubernetes). Подход DDD нередко используется для определения границ сервисов. SRE-практики (Site Reliability Engineering) также актуальны: при большом числе сервисов требуется глубокое внимание к надёжности и мониторингу (пример – Netflix, где команда SRE («Core») поддерживает отказоустойчивость облачной системы).

- **Отказоустойчивость и масштабирование:** микросервисы обеспечивают высокую отказоустойчивость за счёт изоляции: при падении одного сервиса остальная система продолжает работать. Горизонтальное масштабирование легче реализовать – каждый сервис можно масштабировать независимо (с помощью контейнеров и оркестрации). Разделение по сервисам позволяет эластично перераспределять ресурсы и быстрее откатывать неудачные версии.

- **Примеры:** такие крупные компании, как Amazon, Netflix, Uber и Twitter, успешно используют микросервисы для своих высоконагруженных систем. Amazon ушёл от монолита к сотням микросервисов (управление заказами, инвентарём, рекомендациями и т.д.), что ускорило релизы и упростило масштабирование. Netflix перешёл на микросервисную архитектуру (сеть контейнеров в AWS, собственная CDN) для обработки пикового трафика потокового видео. Uber применяет микросервисы для

биллинга, диспетчеризации и других функций, позволяя менять и тестировать отдельные части без риска для остальных. Twitter отказался от монолита, чтобы разным командам позволить выбирать собственный технологический стек (Scala, Java и т.д.). Компания Atlassian (Jira/Confluence) тоже перешла с локального монолита на AWS и постепенно разбивает приложение на микросервисы.

Событийно-ориентированная архитектура (EDA)

Событийно-ориентированная архитектура (Event-Driven Architecture, EDA) базируется на асинхронной модели взаимодействия: компоненты системы обмениваются «событиями» (сообщениями об изменениях состояния) через брокеры или шины сообщений. Каждый сервис реагирует на входящее событие без жёсткой привязки к отправителю, что значительно снижает связанность компонентов. Такая архитектура обеспечивает высокую масштабируемость и гибкость: благодаря асинхронной обработке системы могут параллельно выполнять множество операций, а независимость компонентов упрощает наращивание ресурсов при возросшей нагрузке. EDA часто применяется вместе с микросервисами и DDD: внутри bounded context сервисы обмениваются доменными событиями.

- **Особенности:** публикация/подписка на события через брокеры (Kafka, RabbitMQ и др.), полная развязка продюсеров и потребителей. События обрабатываются в реальном времени, что позволяет системе быстро реагировать на изменения (минимальная задержка между событием и действием).

- **Методологии разработки:** EDA хорошо сочетается с Agile- и DevOps-подходом, поскольку требует гибкости при проектировании взаимодействий и быстрого развертывания. Часто используют связанные паттерны (CQRS, sagas) для согласования данных. Для контроля над

сложностью при большой событийной нагрузке применяют SRE-практики (тесты устойчивости, мониторинг событийных потоков).

- **Отказоустойчивость и масштабирование:** слабая связность компонентов повышает отказоустойчивость – сбой одного сервиса не мешает работе остальных, а брокеры гарантируют доставку событий даже при сбое некоторых узлов. Система легко масштабируется горизонтально: новые экземпляры сервисов можно «подключать» к тем же каналам событий и балансировать обработку. EDA подходит для архитектур с **eventual consistency**, когда мгновенная консистентность не критична, а важнее скорость и масштаб.

- **Примеры:** системы с высокими требованиями к масштабируемости и реальному времени часто строят на EDA. Классическим примером является Netflix – он использует потоковую обработку событий (Kafka/Flink) для телеметрии и рекомендаций. Похожим образом Uber обрабатывает события изменения геолокации водителей и запросы пользователей в реальном времени. EDA широко применяется в интернет-торговле: размещение заказа генерирует цепочку событий (обновление запасов, оповещение логистики, подтверждение клиенту) без прямой синхронной связи модулей. Такие компании, как Netflix и Uber, отмечают высокую отказоустойчивость и способность их платформ выдерживать пиковую нагрузку именно благодаря событийной архитектуре.

CQRS и Event Sourcing

CQRS (Command Query Responsibility Segregation) и Event Sourcing — это архитектурные паттерны, часто используемые в сочетании с EDA и микросервисами. **CQRS** разделяет операции изменения состояния (команды) и чтения данных (запросы), что позволяет оптимизировать каждую часть независимо. **Event Sourcing** предполагает хранение истории всех изменений

состояния в виде последовательности событий, что даёт возможность воспроизводить текущие данные и вести полный аудит. Совместное применение этих паттернов позволяет значительно повысить производительность и масштабируемость системы: команды могут обрабатываться асинхронно, а проекции для чтения строятся по событиям.

- **Особенности:** CQRS устраняет конфликты при одновременной работе на запись и на чтение, а Event Sourcing сохраняет неизменяемый журнал событий. Это даёт дополнительные возможности (аудит, откат состояния, валидация старых транзакций) и повышает надёжность системы. Однако такие паттерны увеличивают сложность: нужны дополнительные механизмы согласованности и управление зависимостями между данными.

- **Методологии разработки:** CQRS/ES хорошо вписываются в DDD и EDA-подходы. Чёткое определение агрегатов и доменных событий (BDD, DDD) помогает сформулировать команды и события. Часто используется DevOps-практика контейнеризации для развёртывания event-store и сервисов проекций; SRE-метрики применяются для контроля задержки репликации событий.

- **Отказоустойчивость и масштабирование:** разделение на логическую модель команд и проекции для чтения позволяет масштабировать операции чтения независимо от записи. Хранение событий гарантирует сохранность данных: даже при сбое компонентов история не теряется и состояние можно восстановить. Event Sourcing легко комбинируется с EDA: события публикуются в очередь и обрабатываются потребителями (проекциями).

- **Примеры:** CQRS+Event Sourcing применяются в системах с жёсткими требованиями к аудиту и истории операций (финансовые системы, бухгалтерия, торговые площадки). По оценкам, подобные схемы

используются лишь в узком диапазоне (например, ~1 % приложений) высокого уровня сложности. Среди известных примеров – банковские платформы и торговые системы, где критична последовательность событий и высокая нагрузка (например, биржи и платёжные процессоры).

Cloud Native архитектура

Cloud Native – это подход к архитектуре приложений, при котором все возможности облачных технологий учитываются «с нуля» во всех аспектах системы: от архитектуры и безопасности до процессов DevOps. Такое приложение разрабатывают под среду облака (публичного или частного) и ориентируют на облачные сервисы, контейнеризацию и автоматизацию. Основные принципы Cloud Native включают использование контейнеров (Docker), оркестрацию (Kubernetes), инфраструктуру как код и бессерверные сервисы. Это позволяет автоматически масштабировать приложение по требованию и быстро развертывать новые версии.

- **Особенности:** построение по 12-факторам (изолированные сервисы, конфиг как код, логи как поток и т.д.), контейнеризация и декомпозиция на микросервисы. Облачные сервисы (IaaS/PaaS) применяются для автоматического распределения ресурсов и обеспечения отказоустойчивости. Непрерывная поставка (CI/CD) и автоматическое тестирование являются обязательной частью процесса разработки.

- **Методологии разработки:** Cloud Native тесно интегрирован с DevOps и Agile-практиками. DevOps-инструменты позволяют быстро разворачивать контейнеры и обновлять компоненты без простоя. Подход DDD также хорошо вписывается – границы сервисов проектируются согласно функциональным областям, а SRE (Site Reliability Engineering)-философия отвечает за мониторинг и поддержание стабильности в облачной среде.

Культура «Infrastructure as Code» обеспечивает предсказуемость развёртывания и воспроизводимость окружения.

- **Отказоустойчивость и масштабирование:** облачная среда изначально рассчитана на отказоустойчивость – приложения спроектированы так, чтобы любая сбойная нода автоматически заменялась, а нагрузка перераспределялась. Масштабирование происходит динамически: контейнеры запускаются или останавливаются по потребности, ресурсы автоматически перераспределяются. Например, Netflix как типичный cloud-native-пример использует контейнерный оркестратор (Titus/Kubernetes) и несколько регионов AWS для достижения нулевого даунтайма.

- **Примеры:** ведущие технологические компании строят свои системы как cloud-native. Архитектура Netflix – это облачная система на основе микросервисов и контейнеров с собственным CDN, которая обеспечивает 99,99 % аптайма. Компания полностью перешла в AWS, отказавшись от монолита, внедрила практики Chaos Engineering и DevOps для обеспечения устойчивости и гибкости. Аналогично, Google (GCP), Amazon (AWS) и другие провайдеры активно продвигают cloud-native подходы (Kubernetes, Lambda/FaaS, облачные базы данных) для высоконагруженных решений. Cloud Native архитектура даёт системам эластичность, автоматическое масштабирование и быстрое время вывода новых функций на рынок

Сравнительная таблица подходов

Подход	Масштабируемость	Отказоустойчивость	Гибкость	Скорость разработки и	Применимость к ВН-системам

Waterfall (классика)	Низкая (сложно оперативно масштабировать)	Средняя (жёсткое планирование, много тестирования)	Низкая (при изменениях – большие задержки)	Низкая (требуется долгое планирование)	Подходит для стабильно запланированных проектов
Agile	Высокая (фреймворки Agile/SAFe поддерживают масштабирование команд)	Высокая (быстрый отклик на сбои, постоянное тестирование)	Очень высокая (итеративная адаптация)	Высокая (короткие спринты, быстрая доставка)	Хорошо подходит к динамическим ВН-проектам (Netflix, Amazon и др.)
DevOps	Очень высокая (CI/CD и IaC позволяют быстро масштабировать окружения)	Очень высокая (постоянный мониторинг и автоматизация)	Высокая (автоматизация убирает бюрократию)	Очень высокая (непрерывный деплой, быстрые итерации)	Идеально для ВН: обеспечивает быстрые выпуски и устойчивость (Netflix, Yandex)

SRE	Очень высокая (системы самовосстаиваются, число SRE растёт сублинейно с нагрузкой)	Очень высокая (приоритет SLO/SLI, автоматизация)	Высокая (DevOps-культура + инженерные практики)	Высокая (постоянные улучшения, быстрый фидбек)	Ключевой подход для ВН (Google, Yandex). Обеспечивает надёжность в масштабе
DDD (Domain-Driven)	Высокая (разделение на контексты позволяет масштабировать области)	Высокая (чёткое разделение доменов уменьшает связанность)	Очень высокая (ясные границы доменов, независимые команды)	Повышенная (меньшая сложность кода)	Уместен для сложных бизнес-приложений (финансы, e-commerce). Снижает сложность и ускоряет разработку
Монолит	Низкая (масштабируется только целиком,	Средняя/низкая (ошибка в модуле может повредить	Низкая (ограничен единым стеком технологий)	Высокая (на старте – единая кодовая база)	Подходит для небольших систем. При росте нагрузки становится узким местом

	без деления)	весь сервис)			
Мик росе рвис ы	Очень высокая (каждый сервис масштабир уется отдельно)	Очень высокая (отказ одного сервиса изолирован , система остаётся работоспос обной)	Очень высокая (независи мые сервисы, разные технологии)	Высокая (благодаря CI/CD и декомпозиц ии)	Широко используются в ВН-системах (Netflix, Amazon, Yandex, VK и др.). Позволяют достичь гибкой масштабируемо сти
EDA (события)	Очень высокая (распредел ённая обработка событий, параллельн ые потребител и)	Очень высокая (если один компонент падает, другие обрабатыва ют события)	Очень высокая (слабая связаннос ть компонен тов)	Средняя (сложнее проектиров ать, но независимо сть ускоряет интеграции)	Подходит для систем реального времени и поточковой обработки (LinkedIn, Yandex, VK), обеспечивает масштабируемо сть и устойчивость

CQRS + Event Sourcing	Очень высокая (горизонтальное масштабирование чтения/записи, масштабируемый лог событий)	Очень высокая (каждое изменение сохраняется в логе, что позволяет восстанавливать состояние)	Высокая (раздельные модели чтения и записи)	Низкая (значительная архитектурная сложность, высокий «когнитивный» overhead)	Применимы в системах с интенсивной записью/чтением и требованиями к аудиту (финансы, биллинг). Позволяют выдерживать высокую нагрузку и полный учёт изменений
------------------------------	--	--	---	---	---

Каждый из перечисленных подходов находит своё практическое применение. Например, **Netflix** после проблем с монолитом перешёл на микросервисы и внедрил DevOps, что позволило компании масштабироваться в облаке и раскатывать новые фичи тысячи раз в день[16]. **Google** формализовал практику SRE для управления своими глобальными сервисами, что дало сублинейный рост потребности в SRE-инженерах и высокую надёжность сервисов[18] **Яндекс** и **VK** активно используют микросервисы, асинхронные шины (Kafka) и практики DevOps/SRE: например, в Yandex.Cloud DevOps/SRE-инженеры отвечают за доставку кода и стабильность сервисов, а VK развивает собственные решения для балансировки и защиты на уровне событий. Таким образом, выбор подхода

зависит от конкретных требований: для очень больших и динамичных систем предпочтительны микросервисы, DevOps и SRE, а при строгой регламентации или в простых проектах могут применяться Waterfall или монолиты.

1.4. Методологические рекомендации

Проектирование высоконагруженных систем требует сочетания гибкости разработки и строгих практик обеспечения надёжности. Современные подходы ориентированы на адаптацию процессов под резкое изменение нагрузки и сложность предметной области. Российский рынок highload-инфраструктуры находится на этапе трансформации: под влиянием санкций, импортозамещения и роста спроса компании вынуждены пересматривать методы построения ИТ-архитектуры. В таких условиях методологии Agile, DevOps, DDD, SRE и другие должны быть адаптированы к особенностям масштабируемых систем, при этом учитываться региональные ограничения (локализация данных, отечественные облачные платформы и ПО).

Рекомендации для разных категорий специалистов

- **Архитекторам:** необходимо закладывать масштабируемость и отказоустойчивость на этапе проектирования. Следует применять Domain-Driven подход для разбивки системы на модули и выделения узких мест. Рекомендуется планировать архитектуру с запасом – предусматривать горизонтальное масштабирование, кэширование и распределённые кластеры. Как отмечают эксперты, **планирование архитектуры под будущий рост нагрузки – неотъемлемая часть успеха** современных систем. Архитекторы должны выбирать архитектурный стиль (монолит или микросервисы) исходя из ожидаемой нагрузки: для систем со стремительным ростом запросов микросервисы и cloud-native решения вкупе с SRE практиками предпочтительнее классического монолита. При этом необходимо предусмотреть возможность интеграции с отечественными технологиями

(PostgreSQL, Tarantool, ClickHouse и др.) и хранилищами на локальных облаках.

- **Разработчикам:** важно соблюдать высокие стандарты качества кода и автоматизировать тестирование. Рекомендуется применять практики *Shift Left* – перенос тестирования на ранние стадии (unit-тесты, TDD). Парное программирование и код-ревью помогают выявлять проблемы с производительностью до релиза. Разработчики должны внедрять встроенное логирование и метрики (observability) для последующего мониторинга производительности в бою. При работе с критичными компонентами нужно учитывать требования к параллелизму и асинхронности, а также ограничения по памяти и ЦПУ. Следует активно использовать контейнеризацию и инструменты DevOps (Docker, Kubernetes, CI/CD) для воспроизводимости окружения и ускорения релизов.

- **Менеджерам:** необходимо выбирать методы разработки и операционной поддержки, соотносясь с бизнес-целями и масштабом бизнеса. На ранних этапах (стартап) правильнее делать ставку на быструю доставку ценности пользователям (Agile/DevOps) и минимальные начальные вложения. По мере роста продуктов руководители должны инвестировать в инфраструктуру надёжности – внедрять SRE-команду или практики SRE (определять SLO, error budget, postmortems). Менеджерам следует контролировать баланс между скоростью и стабильностью релизов: например, используя методологию SRE, они могут вводить ограничения на количество изменений («бюджет ошибок»), при достижении которого команда переключается с разработки новых фич на исправление дефектов. Руководители также обязаны обеспечить соблюдение регуляторных требований: это означает использование одобренных отечественных облаков и сервисов (Yandex.Cloud, Сбертех, VK.Cloud и т.п.), а также соответствие

законам о локализации данных. Для повышения надёжности менеджерам полезно внедрять практики непрерывного анализа рисков и оценки производительности (DevOps-метрики, SLO).

- **Исследователям и аналитикам:** стоит фокусироваться на изучении новых технологий и практик, способных повысить устойчивость систем. В числе перспективных направлений – применение машинного обучения для прогнозирования пиковых нагрузок и автоматического масштабирования ресурсов. Исследователи могут разрабатывать модели для оптимизации распределённого кеширования, анализировать эффективность разных стратегий балансировки нагрузки и изучать алгоритмы самовосстановления. Также важна адаптация международного опыта под российские реалии: исследования должны учитывать ограничения на иностранное ПО и локальное хранение данных, предлагая сценарии высоконагруженных решений с использованием отечественных компонентов и гибридных облачных архитектур.

Учет масштаба и зрелости организаций

Выбор методологии также зависит от размера и зрелости компании. **Стартапы** обычно имеют небольшие команды и не могут позволить себе сложные процессы: здесь предпочтительны Agile-подходы и базовые DevOps-инструменты (Git, лёгкие CI/CD) для максимально быстрой поставки MVP. Как правило, стартапы не фокусируются на SRE-практиках с первого дня, полагаясь на публичное облако и упрощённую инфраструктуру. **Средние компании** начинают испытывать растущие нагрузки и должны постепенно усиливать процессы: вводить формальные DevOps-конвейеры, заботиться о мониторинге и резервировании. Используют микросервисные или гибридные архитектуры, вводят первые SRE-метрики (SLA/SLO) и уделяют внимание бизнес-аналитике для прогнозирования нагрузки. **Крупные предприятия**

оперируют экстремальными нагрузками и жёсткой регуляторикой. Они внедряют полные практики SRE (выделенные SRE-команды, чёткие бюджетные ограничения), развивают распределённые (в т.ч. мультирегиональные) системы и используют комплексные решения для отказоустойчивости (например, резервные кластеры, технологии disaster recovery). В таких организациях часто формально разделяют ответственность между разработчиками и техподдержкой, внедряют культуру DevOps на уровне нескольких команд, но при этом именно инженерия надёжности выходит на первый план. Известно, что крупные сервисы (например, Netflix) не могут обойтись без SRE-дисциплины для обеспечения стабильности стриминга при миллионах одновременных подключений, тогда как agile-ориентированные стартапы чаще выбирают упрощённые DevOps-практики (с фокусом на быструю доставку функций). При этом даже в больших компаниях следует поддерживать гибкость: сочетание Agile-смотрит со строгими SRE-метриками позволяет найти оптимальный баланс скорости и надёжности.

Сравнительная таблица применения методологий

Архитектурны й тип / Организацион ный масштаб	Стартапы	Средние компании	Крупные предприятия

Монолит	Небольшой монолитный прототип; активно Agile/Scrum; минимальный DevOps (правка кода напрямую на сервере или простые CI); нагрузка обычно низкая.	Стабилизация с помощью DevOps: настроены CI/CD, резервные копии; постепенный рефакторинг на модули; начинают использовать DDD-моделирование для разделения обязанностей.	Монолит становится «legacy»: планируется рефакторинг в микросервисы или облачные сервисы; обязательны SRE/DevOps-инструменты (канары, blue-green); использование отечественных СУБД/облаков для соответствия регуляциям.
----------------	--	--	--

Микросервисы	Микросервисы с упрощённой инфраструктурой (Docker, базовые оркестраторы); Agile с частыми итерациями; DevOps для автоматических сборок; DDD для разграничения микросервисов.	Полноценный DevOps: CI/CD, оркестрация (Kubernetes), мониторинг сервисов; Domain-Driven подход обеспечивает устойчивость доменных границ; начинают вводить SRE-практики (метрики отказоустойчивости, SLIs/SLOs).	Крайне зрелая DevOps-система: мульти-кластерные облака, SRE-инженеры контролируют стабильность (авторотация инцидентов, budget), автоматическое горизонтальное масштабирование. Используются отечественные PaaS/IaaS (Yandex.Cloud, СберCloud) и СУБД (PostgreSQL, Tarantool).
---------------------	--	--	--

Cloud-native	Активное использование публичных облаков (VM, контейнеры, serverless); IaC (Terraform/Ansible) для быстрого старта; фокус на MVP. DevOps встроено «по умолчанию», нагрузка переносится на облако.	Гибридное облако: комбинируют локальные ЦОД и облака; автоматическое масштабирование (autoscaling groups); DevOps+SRE: детальная телеметрия и алёртинг; DDD и микросервисы с событийной архитектурой (Kafka и др.).	Полностью распределённая облачная инфраструктура (Private/Public), сложная микросервисная архитектура; развитые SRE-практики: отказоустойчивость, ML-прогнозирование пиков (AI-оптимизация); жёсткие SLA и избыточность; обязательна локализация данных и импортозамещающие сервисы.
---------------------	---	---	--

Таким образом, при проектировании высоконагруженных систем рекомендуется **гибридный подход**: сочетать гибкие методологии разработки (Agile/DevOps) с инженерией надёжности (SRE, DDD и др.), адаптируя их под конкретные условия системы и организационные требования.

Agile-методы обеспечивают скорость реакции на изменения, а *DevOps/SRE* – масштабируемость и устойчивость систем к сбоям. Предметно-ориентированное проектирование (DDD) закладывает модульность и понятность архитектуры, упрощая дальнейшее расширение продукта. Учитывая российские реалии (санкции, новые законы о хранении данных и «импортозамещение»), предприятия должны опираться на отечественные платформы и ПО. В результате для эффективных высоконагруженных систем необходимо найти компромисс: гибкость процессов разработки должна сочетаться с систематической работой над надёжностью и производительностью. В конечном счёте именно такое интегрированное решение позволит удовлетворять высокие требования к нагрузке и устойчивости сервисов в современных условиях.

Заключение

В теоретической главе рассмотрены ключевые аспекты проектирования высоконагруженных систем и сформулированы их основные характеристики. В частности, в разделе 1.1 анализ включал основные метрики и параметры таких систем, как высокая нагрузка (большие значения QPS и пропускной способности), отказоустойчивость (резервирование, автоматическое восстановление), масштабируемость (горизонтальная и вертикальная) и ограничения модели CAP (компромисс между согласованностью, доступностью и устойчивостью к разделению). Рассмотренные примеры крупных технологических компаний (Google, Amazon, ВКонтакте, Яндекс) иллюстрируют применение описанных принципов: они обеспечивают высокую пропускную способность и отказоустойчивость за счёт распределённых архитектур, при этом балансируя между согласованностью и доступностью данных в соответствии с теоремой CAP.

В разделе 1.2 проведён обзор методологических подходов к разработке программного обеспечения и их применимости в условиях highload. Показано, что традиционные классические методологии (Waterfall, RUP) обладают формальной структурой и простотой планирования, но часто ограничены низкой гибкостью и реактивностью при появлении новых требований и непредвиденных нагрузок. Гибкие методологии (Agile, Scrum) и практики DevOps ориентированы на итеративную разработку и непрерывную интеграцию, что позволяет быстрее адаптироваться к изменениям требований и нагрузок. Также рассмотрены практики Domain-Driven Design (DDD) и Site Reliability Engineering (SRE), подчёркивающие важность фокусировки на доменной модели и надёжности: DDD помогает структурировать сложную

бизнес-логику, а принципы SRE – поддерживать необходимый уровень доступности и минимизировать количество отказов. В целом подходы Agile, DevOps и SRE оказываются более эффективными для разработки высоконагруженных систем, в то время как Waterfall пригоден для менее критичных по нагрузке проектов.

В разделе 1.3 были подведены итоги по архитектурным подходам. Микросервисная архитектура обеспечивает высокую масштабируемость и независимое развертывание компонентов, что даёт гибкость в управлении нагрузкой и быстрый ввод новых функций, но усложняет мониторинг, отладку и обеспечение согласованности данных между сервисами. Монолитная архитектура проще в разработке и тестировании, но ограничена в масштабировании (обычно вертикальном) и имеет больший риск полного отказа при сбое одного из компонентов. Событийно-ориентированные системы (event-driven) эффективны для обработки больших потоков событий и повышения асинхронности, что улучшает отказоустойчивость и масштабируемость. Cloud-native подходы (контейнеризация, оркестрация, serverless-технологии) обеспечивают гибкое автоматическое масштабирование компонентов и упрощают переносимость. Микросервисные архитектуры обычно хорошо сочетаются с гибкими методологиями разработки и DevOps-подходами, тогда как монолитные системы традиционно разрабатываются в рамках более формальных процессов.

В разделе 1.4 сформулированы методологические рекомендации по проектированию высоконагруженных систем. Выделены принципы адаптации и комбинирования подходов: сочетание итеративных практик разработки (Scrum, Kanban) с интеграцией DevOps-культуры и SRE-инженерии позволяет сохранять надёжность при быстром темпе

изменений. Рекомендовано использовать гибридные модели разработки и непрерывного развертывания для быстрой проверки гипотез и оперативного выпуска улучшений. Выбор архитектуры определяется ожидаемыми сценариями нагрузки: для систем с экстремально высокой нагрузкой предпочтительны микросервисные и событийно-ориентированные архитектуры, тогда как для проектов со средней или предсказуемой нагрузкой может быть оправдана упрощённая монолитная структура. Рекомендовано постоянно отслеживать ключевые метрики (latency, throughput, error rate, QPS) для своевременного обнаружения узких мест и принятия корректирующих мер. Среди перспективных направлений развития практик выделяются усиление роли SRE, широкое внедрение облачно-ориентированных платформ и ИИ-инструментов для прогнозирования нагрузки и автоматизации балансировки ресурсов.

В целом проведённый системный анализ демонстрирует, что проектирование высоконагруженных систем требует взвешенного баланса между формальными методологиями и гибкостью, тщательного подбора архитектурных паттернов и постоянного внимания к ключевым метрикам производительности и надёжности.